

Agent State, LangGraph State

Agent State is the information an agent needs to remember and track across its operation.

AKA: Agent's working memory

To simply put, State acts as a shared, evolving data structure that hold the current snapshot of your application, passing context, history, and variable between the node in your workflows.

It includes:

- Conversation history
- Tool call history
- Scratchpad/Reasoning trace
- Task context
- External memory

Example: user = "Search for papers on RAG, summarize the top 3, and then save them to an aggregated file."

```
{
  "goal": "Search papers, summarize top 3, save to file",
  "steps_done": ["search_arxiv(query='RAG')"],
  "tool_results": {"search": ["paper1", "paper2", "paper3"]},
  "messages": [...],
  "next_step": "summarize each paper"
}
```

So without the state, it does not know that the papers have already been found

What it has to do with the StateBackend

When we pass the StateBackend() to an agent, you are telling the agent where to store and reload this state, so it can pause, resume, or survive the crash or any errors.

- If it is in-memory, the state is gone when the process end.
- If it is a DB backend, the agent can pick up exactly where it left off.

So whenever we use the StateBackend, the memory goes into the LangGraph, but it depend on checkpointers you have attached it to the graph (InMemorySaver, PostgresSaver, etc) is what actually persist this data. Without the checkpointers assigned, nothing will be saved after the past run.